Assignment (Term Paper)

Statistical Programming, WS Term 2025/2026

2026-03-20

Team members

Niclas Schwarzbach (7962944)
Eric Ollanescu-Orendi (7959293)
Jan Gabske (7307849)
Abdel Rahman Obaisi (7168318)

Section 1: Data Wrangling and Spatial Exploration

This notebook covers the cleaning, transformation, and first exploration of the Berlin Airbnb data. The aim is to prepare a usable dataset and describe the main price patterns in a clear way.

```
# Task 1.1

import numpy as np
import pandas as pd

listings = pd.read_csv("../Daten/listings_berlin.csv")

print(f"Number of rows: {len(listings)}")
print(f"Number of columns: {len(listings.columns)}")
missing_count = listings.isna().sum()
missing_share = listings.isna().mean().map("{:.2%}".format)

Table_1_1 = pd.DataFrame({"Number of missings": missing_count, "Share of missings": missing_share})
Table_1_1.head(20)
```

```
Number of rows: 14274
Number of columns: 20
```

	Number of missings	Share of missings
price	5010	35.10%
neighbourhood_cleansed	0	0.00%
latitude	0	0.00%
longitude	0	0.00%
room_type	0	0.00%
bedrooms	2024	14.18%
bathrooms_text	16	0.11%
accommodates	0	0.00%
amenities	0	0.00%
host_since	14	0.10%
host_is_superhost	164	1.15%
host_listings_count	14	0.10%
host_response_rate	5018	35.15%
review_scores_rating	3314	23.22%
review_scores_cleanliness	3314	23.22%
number_of_reviews	0	0.00%
reviews_per_month	3314	23.22%
minimum_nights	0	0.00%
availability_365	0	0.00%
instant_bookable	0	0.00%

```
# Task 1.2
# Sort the table by missing-value share and display the top three columns.
print("The following three variables have the highest missing-value rates:")
Table_1_1.sort_values("Share of missings", ascending=False).head(3)
```

```
The following three variables have the highest missing-value rates:
```

	Number of missings	Share of missings
host_response_rate	5018	35.15%
price	5010	35.10%
reviews_per_month	3314	23.22%

The three variables with the largest missing-value shares need different treatment.

- **Price** should clearly be kept, since it is the main variable for later analysis. Missing prices cannot be imputed in a convincing way here, so we keep the column and only drop observations when price is required in a specific step.
- **Host response rate** may still contain useful host-level information. We therefore keep the column for now instead of dropping it immediately.
- **Reviews per month** is less essential because the dataset already includes the total number of reviews. For our later analysis it is not central, so this variable could be dropped without losing much information.

Overall, we prefer to keep potentially relevant variables unless the later analysis shows that they are not needed.

```
# Task 1.3
# Convert the price column from string format to numeric values.
def clean_price(price_str):
    """Convert a price string to a numeric value."""

    if pd.isna(price_str):
        return np.nan
    price_str_clean = str(price_str).replace("$", "").replace(',', '')
    try:
        price_number = float(price_str_clean)
        if price_number > 0:
            return price_number
        return np.nan
    except ValueError:
        return np.nan

# Apply the cleaning function to the full price column.
listings_1_3 = listings.copy()
listings_1_3["price"] = listings_1_3["price"].apply(clean_price)
listings_1_3["price"].describe().loc[['min', '50%', 'mean', 'max']]
```

```
min         5.000000
50%       104.000000
mean      201.240393
max     50000.000000
Name: price, dtype: float64
```

```
# Task 1.4
listings_1_4 = listings_1_3.copy()
# We first inspect the unique entries in bathrooms_text if needed.
# listings_1_4["bathrooms_text"].unique()

# Convert the bathroom information from text into a numeric representation.
# Rule 1: "private" is redundant for the numeric count.
# Rule 2: "shared" is stored in a separate boolean column.
# Rule 3: a half bath is coded as 0.5.

# Store shared-bath information.
listings_1_4["Shared_bath_status"] = listings_1_4["bathrooms_text"].str.contains("shared", case=Falsc
# Recode half baths.
listings_1_4["bathrooms_text"] = listings_1_4["bathrooms_text"].str.replace("Half-Bath", "0.5", case=
# Convert the remaining text to numeric values.
listings_1_4["bathrooms_text"] = pd.to_numeric(
    listings_1_4["bathrooms_text"].str.replace({
        "baths": "",
        "shared": "",
        "private": "",
        "bath": "",
```

```
        " ": ""
    }, case=False)
)
listings_1_4.rename(columns={"bathrooms_text": "bathrooms_number"}, inplace=True)
listings_1_4[["bathrooms_number", "Shared_bath_status"]].head()
print(listings_1_4.isna().sum())
```

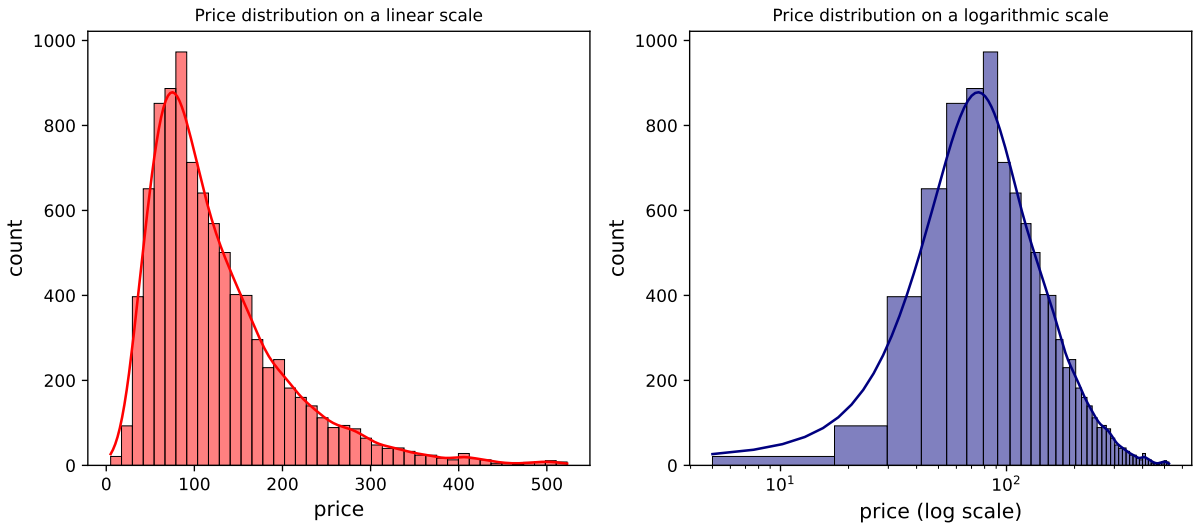
```
price                        5010
neighbourhood_cleansed          0
latitude                        0
longitude                       0
room_type                       0
bedrooms                     2024
bathrooms_number               16
accommodates                    0
amenities                       0
host_since                     14
host_is_superhost             164
host_listings_count            14
host_response_rate           5018
review_scores_rating         3314
review_scores_cleanliness    3314
number_of_reviews               0
reviews_per_month            3314
minimum_nights                  0
availability_365                0
instant_bookable                0
Shared_bath_status              0
dtype: int64
```

```
# Task 1.5
listings_1_5 = listings_1_4.copy()
# Convert "t" and "f" to boolean values.
listings_1_5["instant_bookable"] = listings_1_5["instant_bookable"] == "t"
listings_1_5["host_is_superhost"] = listings_1_5["host_is_superhost"] == "t"
# Create host tenure in years and drop the original date column.
listings_1_5["host_tenure_years"] = 2025 - pd.to_datetime(listings_1_5["host_since"]).dt.year
listings_1_5 = listings_1_5.drop(columns="host_since")
listings_1_5.columns
```

```
Index(['price', 'neighbourhood_cleansed', 'latitude', 'longitude', 'room_type',
       'bedrooms', 'bathrooms_number', 'accommodates', 'amenities',
       'host_is_superhost', 'host_listings_count', 'host_response_rate',
       'review_scores_rating', 'review_scores_cleanliness',
       'number_of_reviews', 'reviews_per_month', 'minimum_nights',
       'availability_365', 'instant_bookable', 'Shared_bath_status',
       'host_tenure_years'],
      dtype='str')
```

```
# Task 1.6
import matplotlib.pyplot as plt
import seaborn as sns

log_prices = np.log(listings_1_5["price"])
upper_bound = np.exp(np.mean(log_prices) + 2.33 * np.std(log_prices))
# A z-value of 2.33 covers about 99% of the log-price distribution.

listings_1_6 = listings_1_5.loc[listings_1_5["price"] <= upper_bound].copy()
```

```
fig, ax = plt.subplots(1, 2, sharey=False, figsize=(10, 4.5))

sns.histplot(data=listings_1_6, x="price", kde=True, ax=ax[0], log=False,
             bins=round(10*np.log10(len(listings_1_5))), color="red", stat="count")
ax[0].set_xlabel("price", fontsize=12)
ax[0].set_ylabel("count", fontsize=12)
ax[0].set_title("Price distribution on a linear scale", fontsize=10)

sns.histplot(data=listings_1_6, x="price", kde=True, ax=ax[1], log=False,
             bins=round(10*np.log10(len(listings_1_5))), color="navy", stat="count")
ax[1].set_xlabel("price (log scale)", fontsize=12)
ax[1].set_ylabel("count", fontsize=12)
ax[1].set_xscale("log")
ax[1].set_title("Price distribution on a logarithmic scale", fontsize=10)
plt.tight_layout()
plt.show()
```



When based on the full dataset, the histogramm on both the linear and the log scale are heavily dominated by outliers. The main body of the distribution would be compressed into such a narrow range, that an analysis becomes impossible. We chose to bypass that issue by introducing an upper bound to the price variable in our dataset. We calculated the bound to include 99% of listings based on the assumption of a log normal distribution. The resulting plots comfirm this assumption, as the log scale plot ist roughly normally distributed, while the linear scale plot is heavily right-skewed.

For later statistical analysis, the logarithmic representation is the better choice. Prices appear roughly log-normal, and the log scale reduces the influence of extreme values. This makes the distribution more stable and better suited for regression-based work.

```
# Task 1.7
# Group by neighbourhood and compute summary statistics for price and ratings.
table_1_7 = listings_1_5.groupby("neighbourhood_cleansed").agg({
    "price": ["mean", "median", "count"],
    "review_scores_rating": "mean"
}).copy()

# Round selected values and sort by median price.
table_1_7 = table_1_7.round({("price", "mean"): 1, ("review_scores_rating", "mean"): 1})

table_1_7.head(3)
# The three most expensive areas in this table are Haselhorst, West 5, and Schmargendorf.
```

neighbourhood_cleansed	price mean	median	count	review_scores_rating mean
Haselhorst	216.2	236.5	6	4.9
West 5	381.5	203.0	11	4.8
Schmargendorf	170.6	172.0	34	4.7

```
# Task 1.8
listings_1_8 = listings_1_5.copy()

def trimmed_mean(x, pct):
    """Calculate the trimmed mean after removing pct percent from both tails."""
    if not (isinstance(pct, (int, float)) and 0 <= pct < 50):
        raise ValueError("pct must be a number between 0 and 50")
    try:
        x = np.asarray(x, dtype=float)
    except (TypeError, ValueError):
        raise TypeError("x must be an array-like object containing numeric values")
    if x.ndim != 1:
        raise ValueError("x must be one-dimensional")
    bound = int(len(x) * pct / 100)
    if bound == 0:
        return np.mean(x)
    if 2 * bound >= len(x):
        raise ValueError("pct too large for the size of x")
    x = x[~np.isnan(x)]
    x = np.sort(x)
    mean_trimmed = x[bound:-bound]
    return np.mean(mean_trimmed)

price_mean_trimmed = trimmed_mean(listings_1_8["price"], 5)
price_mean = listings_1_8["price"].mean()
print(f"The 5% trimmed mean of price is: {price_mean_trimmed:.2f}. The arithmetic mean of price is: 

print("The trimmed mean is lower because a small number of very expensive listings pull the arithmet
```

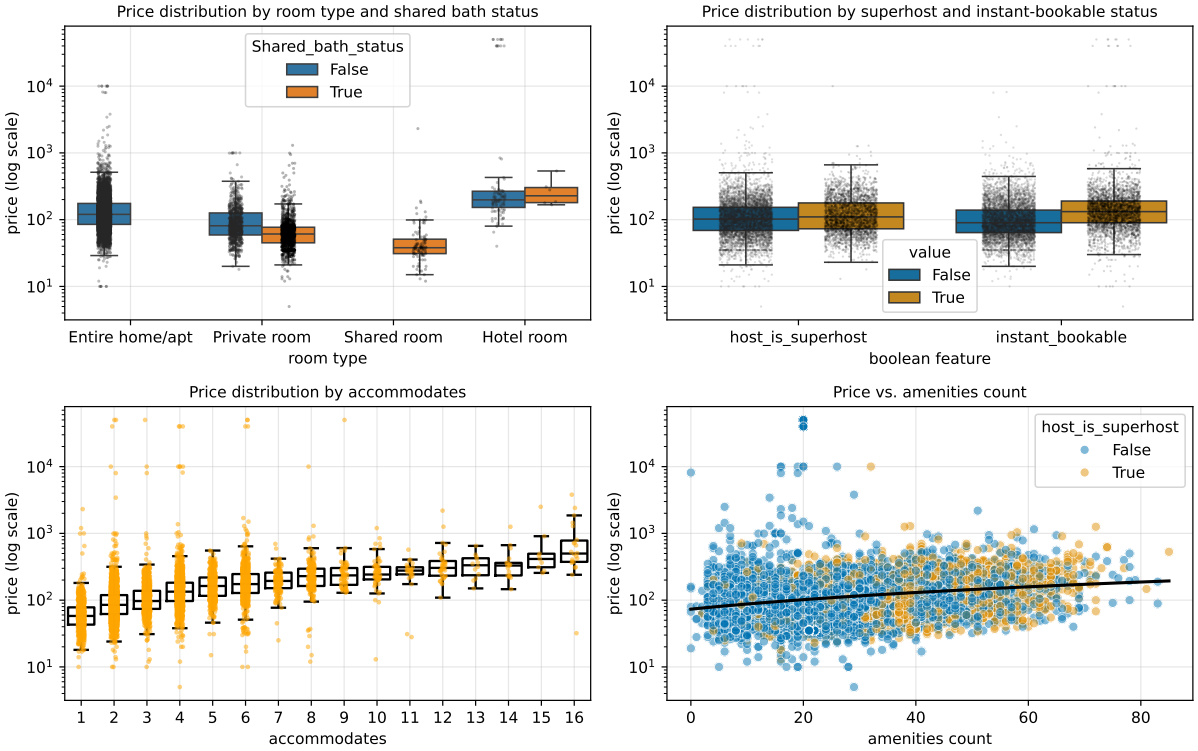
```
The 5% trimmed mean of price is: 116.85. The arithmetic mean of price is: 201.24. This is an increas
The trimmed mean is lower because a small number of very expensive listings pull the arithmetic mean
```

```
#To make the information in the amenities column available to our analysis, we transform the list li
#The existence of a dedicated function with this purpose within the ast library was brought to our a
import ast

# Parse the amenities column from string representation to a Python list.
def clean_amenities(amenities_str):
    if pd.isna(amenities_str):
        return []
    try:
        amenities = ast.literal_eval(amenities_str)
        return [a.strip() for a in amenities if isinstance(a, str)]
    except (ValueError, SyntaxError):
        return []
# We apply the function to our dataset and calculate the number of entries per list.
listings_1_8["amenities_list"] = listings_1_8["amenities"].apply(clean_amenities)
listings_1_8["amenities_count"] = listings_1_8["amenities_list"].apply(len)
# do the same for the full dataset from 1_5
listings_1_5["amenities_list"] = listings_1_5["amenities"].apply(clean_amenities)
listings_1_5["amenities_count"] = listings_1_5["amenities_list"].apply(len)
```

```
listings_1_8[["amenities", "amenities_list", "amenities_count"]].head(3)
```

	amenities	amenities_list	amenities_co
0	["Cooking basics", "Wine glasses", "Crib", "Ho...	[Cooking basics, Wine glasses, Crib, Hot water...	44
1	["Cooking basics", "Crib", "Stove", "Elevator"...	[Cooking basics, Crib, Stove, Elevator, Luggag...	32
2	["Ethernet connection", "Dishes and silverware...	[Ethernet connection, Dishes and silverware, C...	14

```
# Task 1.9
# we load the full data set from 1_5, so we can see the outliers in the plots.
listings_1_9 = listings_1_5.copy()

fig, ax = plt.subplots(2, 2, figsize=(11, 7), dpi=200)

sns.boxplot(data=listings_1_9, x="room_type", y="price", hue="Shared_bath_status", showfliers=False,
sns.stripplot(data=listings_1_9, x="room_type", y="price", hue="Shared_bath_status", dodge=True, pal
ax[0,0].set_title("Price distribution by room type and shared bath status", fontsize=10)
ax[0,0].set_xlabel("room type", fontsize=10)
ax[0,0].set_ylabel("price (log scale)", fontsize=10)

df_long = listings_1_9.melt(
    id_vars=["price"],
    value_vars=["host_is_superhost", "instant_bookable"],
    var_name="feature",
    value_name="value")
sns.boxplot(data=df_long, x="feature", y="price", hue="value", showfliers=False, palette="colorblind
sns.stripplot(data=df_long, x="feature", y="price", hue="value", dodge=True, alpha=0.15, size=1.5, j
               palette="dark:black", ax=ax[0,1], log_scale=True, legend=False)
ax[0,1].set_title("Price distribution by superhost and instant-bookable status", fontsize=10)
ax[0,1].set_xlabel("boolean feature", fontsize=10)
ax[0,1].set_ylabel("price (log scale)", fontsize=10)

sns.boxplot(data=listings_1_9, x="accommodates", y="price", ax=ax[1,0], log_scale=True, color="black
sns.stripplot(data=listings_1_9, x="accommodates", y="price", ax=ax[1,0], alpha=0.5, log_scale=True,
ax[1,0].set_title("Price distribution by accommodates", fontsize=10)
ax[1,0].set_xlabel("accommodates", fontsize=10)
ax[1,0].set_ylabel("price (log scale)", fontsize=10)

sns.scatterplot(data=listings_1_9, x="amenities_count", y="price", ax=ax[1,1], alpha=0.5, hue="host_
sns.regplot(data=listings_1_9, x="amenities_count", y="price", ax=ax[1,1], scatter=False, line_kws={
ax[1,1].set_title("Price vs. amenities count", fontsize=10)
ax[1,1].set_yscale("log")
ax[1,1].set_xlabel("amenities count", fontsize=10)
ax[1,1].set_ylabel("price (log scale)", fontsize=10)

for a in ax.flatten():
    a.grid(alpha=0.3)
ax[0,1].xaxis.grid(False)

plt.tight_layout()
plt.show()
```



The first plot shows large price differences across room types. Shared rooms are clearly the cheapest, private rooms come next, and entire homes or hotel rooms tend to be more expensive. Shared bathrooms are optionally available for private rooms and hotel rooms, and listings with shared bathrooms are usually priced lower for private rooms. Hotel rooms with a shared bath show a higher median price than those with private ones, however the sample size for hotelrooms and those with shared bathrooms specifically is very low, so a general trend should not be derived. The second plot suggests that both superhost status and instant booking are associated with higher prices. The pattern is visible in both boxplots, although the differences are not equally strong. The third plot shows a clear positive relationship between the number of accommodated guests and price. Larger listings are generally more expensive. The fourth plot portrais the dataset in relation to price and the number of items listed in the amenities section. Throughout the cluster, a positive relation can be observed, backed up by the robust regression line. Additionaly the figure shows that superhosts tend to list more amenities. A superhost premium beyond the regressed amenities related price increase is not clearly visible.

Task 1.10: The Berlin Airbnb market shows a clear price gradient across listing characteristics. Room type is one of the strongest factors: shared rooms are cheapest, while entire homes and hotel rooms are more expensive. Capacity also matters a lot, since listings for more guests tend to have substantially higher prices. In addition, listings with more amenities are usually more expensive, which suggests that quality and equipment are reflected in pricing. Superhost status and instant booking also seem to be associated with a price premium, although these effects are less pronounced than room type or capacity. Spatially, central areas often combine high listing density with comparatively high prices, but the relationship is not perfect. Some peripheral regions also show high median prices, likely because they contain fewer listings and are therefore more sensitive to individual outliers. Overall, listing size, type, amenities, and location seem to matter most for prices.

```
# Task 1.11: match region information from the Airbnb data to the GeoJSON data

import json
import re   # We import re to check for and remove inconsistent whitespaces.

# We load the geojson file:

with open("../Daten/berlin_bezirke.geojson", "r", encoding="utf-8") as f:
    berlin_geojson = json.load(f)
```

```
print(len(berlin_geojson["features"]))
berlin_geojson["features"][0].keys()

bzr_names = [feature["properties"]["BZR_NAME"] for feature in berlin_geojson["features"]]
geo_df = pd.DataFrame({"region_geo": bzr_names})

airbnb_regions = listings_1_9["neighbourhood_cleansed"].unique()
airbnb_df = pd.DataFrame({"region_airbnb": airbnb_regions})

def clean_region(name):      # We define a cleaning function to normalize the names of both sets
    name = name.lower()
    name = name.replace("str.", "strasse")
    name = name.replace("-", " ")
    name = name.replace("ß", "ss")
    name = name.replace(" / ", "/")
    name = re.sub(r"\s+", " ", name)
    name = name.strip()

    name = name.split("/")[0]
    name = re.sub(r"(west \d+).*", lambda m: m.group(1), name)
    name = re.sub(r"(nord \d+).*", lambda m: m.group(1), name)
    name = re.sub(r"(ost \d+).*", lambda m: m.group(1), name)

    return name

geo_df["clean_name"] = geo_df["region_geo"].apply(clean_region)
airbnb_df["clean_name"] = airbnb_df["region_airbnb"].apply(clean_region)

# We check for perfect matches after applying the cleaning function:

auto_matches = set(geo_df["clean_name"]) & set(airbnb_df["clean_name"])
mapping_auto = {name: name for name in auto_matches}
mapping_auto_v2 = {     # We map the original names of the matches
    airbnb_df.loc[airbnb_df["clean_name"] == name]["region_airbnb"].iloc[0]:
    geo_df.loc[geo_df["clean_name"] == name]["region_geo"].iloc[0]
    for name in auto_matches
}
remaining_airbnb = set(airbnb_df["clean_name"]) - set(mapping_auto.keys())
remaining_geo = set(geo_df["clean_name"]) - set(mapping_auto.values())
print("Remaining Airbnb regions:", sorted(remaining_airbnb)[:3])
print("Remaining GeoJSON regions:", sorted(remaining_geo)[:3])

# For the unmatched names below, we use a manual mapping after checking the cleaned names with the h
mapping_manual = {
    "friedenau": "friedenau ost",
    "gropiusstadt": "gropiusstadt ost",
    "mariendorf": "mariendorf nord",
    "marienfelde": "marienfelde nord",
    "lichtenrade": "lichtenrade nord",
    "tempelhof": "tempelhof nord",
    "mv 1": "mv nord",
    "mv 2": "mv süd",
    "schöneberg nord": "schöneberg nordost",
    "schöneberg süd": "schöneberg südost",
    "kölln. vorstadt": "köllnische vorstadt",
}
mapping_manual_v2 = {   # We map the original names together.
    airbnb_df.loc[airbnb_df["clean_name"] == name_a]["region_airbnb"].iloc[0]:
```

```
    geo_df.loc[geo_df["clean_name"] == name_g]["region_geo"].iloc[0]
    for name_a, name_g in mapping_manual.items()
}
mapping = {**mapping_auto_v2, **mapping_manual_v2}
print(sorted(mapping)[:4])
```

```
143
Remaining Airbnb regions: ['altstadt kietz', 'barstrasse', 'düsseldorfer strasse']
Remaining GeoJSON regions: ['altstadt köpenick', 'britz nord', 'friedenau ost']
['Adlershof', 'Albrechtstr.', 'Alexanderplatz', 'Allende-Viertel']
```

A few unmatched regions remain after the name-based matching. In principle, these cases can be handled more reliably by using the listing coordinates and assigning each point directly to its geographic polygon. This is the better approach because it does not depend on matching different naming conventions.

```
import geopandas as gpd

# Build a GeoDataFrame from the listing coordinates.
listings_1_11 = listings_1_9.copy()

gdf_listings = gpd.GeoDataFrame(
    listings_1_11,
    geometry=gpd.points_from_xy(listings_1_11["longitude"], listings_1_11["latitude"]),
    crs="EPSG:4326"
)

# Load the GeoJSON file.
gdf_bezirke = gpd.read_file("../Daten/berlin_bezirke.geojson")
gdf_bezirke = gdf_bezirke.to_crs(gdf_listings.crs)

# Spatial join: assign each listing to its Bezirksregion.
result = gpd.sjoin(
    gdf_listings,
    gdf_bezirke[["BZR_NAME", "GROESSE_m2", "geometry"]],
    how="left",
    predicate="within"
)

# Copy the matched region information back to the main table.
listings_1_11[["BZR_NAME", "GROESSE_m2"]] = result[["BZR_NAME", "GROESSE_m2"]].values
listings_1_11 = listings_1_11.drop(columns=["neighbourhood_cleansed"])
listings_1_11.head()
```

	price	latitude	longitude	room_type	bedrooms	bathrooms_number	accommodates	amenities
0	105.0	52.53471	13.41810	Entire home/apt	1.0	1.0	2	["Cooking ba
1	135.0	52.53269	13.41805	Entire home/apt	4.0	2.5	7	["Cooking ba
2	75.0	52.54813	13.40366	Entire home/apt	0.0	1.0	1	["Ethernet co
3	32.0	52.49419	13.42166	Entire home/apt	0.0	1.0	2	["Cooking ba
4	202.0	52.53711	13.40888	Entire home/apt	2.0	1.0	4	["Cooking ba

```
# Task 1.12

# For the following analysis and regression tasks, we exclude the outliers by implementing the 99% u
listings_1_12 = listings_1_11.loc[listings_1_11["price"] < upper_bound].copy()

region_stats = (
    listings_1_12
    .groupby("BZR_NAME")
```

```
    .agg(
        listings=("BZR_NAME", "count"),
        median_price=("price", "median")
    )
    .reset_index()
)

import plotly.express as px

fig = px.choropleth(
    region_stats,
    geojson=berlin_geojson,
    locations="BZR_NAME",
    featureidkey="properties.BZR_NAME",
    color="median_price",
    hover_name="BZR_NAME",
    hover_data={"listings": True, "median_price": True},
    title="Median nightly price of Airbnb listings by Bezirksregion in Berlin"
)

fig.update_geos(fitbounds="locations", visible=True)
fig.update_layout(height=550, margin=dict(l=0, r=0, t=50, b=0))
fig.show()

# Task 1.13
listings_1_13 = listings_1_12.copy()
listings_1_13["region_median_price"] = listings_1_13.groupby("BZR_NAME")["price"].transform("median"
listings_1_13["above_local_median"] = listings_1_13["price"] > listings_1_13["region_median_price"]

scatter = px.scatter_geo(
    listings_1_13,
    lat="latitude",
    lon="longitude",
    color="above_local_median",
    hover_data=["price", "region_median_price", "BZR_NAME"]
)

for trace in scatter.data:
    fig.add_trace(trace)
    fig.update_traces(marker=dict(size=5, opacity=0.5), selector=dict(mode='markers'))

fig.update_layout(height=550, margin=dict(l=0, r=0, t=50, b=0), legend={"x": 0.9, "y": 1})
fig.show()
```

```
Unable to display output for mime type(s): text/html

Unable to display output for mime type(s): text/html

Unable to display output for mime type(s): text/html
```

```
# Task 1.14

region_stats.head()

region_stats = region_stats.merge(gdf_bezirke[["BZR_NAME", "GROESSE_m2"]], on="BZR_NAME", how="left"
region_stats["GROESSE_km2"] = (region_stats["GROESSE_m2"] / 1_000_000).astype(float)
region_stats["listings_per_km2"] = (region_stats["listings"] / region_stats["GROESSE_km2"]).astype(fl

fig = px.choropleth(
```

```
    region_stats,
    geojson=berlin_geojson,
    locations="BZR_NAME",
    featureidkey="properties.BZR_NAME",
    color="listings_per_km2",
    hover_name="BZR_NAME",
    hover_data={"listings": True, "median_price": True},
    title="Airbnb listing density by Bezirksregion in Berlin"
)
fig.update_geos(fitbounds="locations", visible=True)
fig.update_layout(height=550, margin=dict(l=0, r=0, t=50, b=0))
fig.show()
```

```
Unable to display output for mime type(s): text/html
```

Listing density is clearly highest in central Berlin. The density map shows a strong concentration of Airbnb activity near the inner city, while most outer areas have far fewer listings per square kilometer. The price map points in a similar direction, but the pattern is less uniform. Central areas are often expensive, yet some peripheral regions also show high median prices. These cases should be interpreted carefully, because regions with only a small number of listings are more affected by individual high-price observations.

Taken together, the maps suggest that centrality matters for both density and price, but more strongly for density. Proximity to major attractions, public transport, and established urban neighborhoods likely explains why the centre dominates the market. At the same time, local factors such as neighborhood image, housing type, or small sample sizes can create high-price pockets outside the centre.

```
listings_1_13.to_csv("listings_1_13.csv")
```

Section 2

```
# Task 2.1
# Load the cleaned data from Section 1 and prepare the regression sample.
import pandas as pd
import numpy as np
listings_2 = pd.read_csv("../Quarto/listings_1_13.csv", index_col="Unnamed: 0")
listings_2["price"] = listings_2["price"].replace(0, pd.NA)

print(listings_2.columns)

listings_2_1 = listings_2[[
    "price",
    "accommodates",
    "latitude",
    "longitude",
    "amenities_count",
    "bedrooms",
    "host_is_superhost",
    "Shared_bath_status",
    "instant_bookable",
    "bathrooms_number"]].copy()
#Drops NA
listings_2_1.dropna(inplace=True)
listings_2_1["price"] = listings_2_1["price"].apply(np.log)
listings_2_1.rename(columns={"price":"log_price"}, inplace=True)
#We have checked that there are no NA values.  No further cleaning is necessary.

#Report the sample size used in the regression.
print(f"The sample size amounuts to: {len(listings_2_1)}")
```

```
Index(['price', 'latitude', 'longitude', 'room_type', 'bedrooms',
       'bathrooms_number', 'accommodates', 'amenities', 'host_is_superhost',
       'host_listings_count', 'host_response_rate', 'review_scores_rating',
       'review_scores_cleanliness', 'number_of_reviews', 'reviews_per_month',
       'minimum_nights', 'availability_365', 'instant_bookable',
       'Shared_bath_status', 'host_tenure_years', 'amenities_list',
       'amenities_count', 'BZR_NAME', 'GROESSE_m2', 'region_median_price',
       'above_local_median'],
      dtype='str')
The sample size amounuts to: 9093
```

The predictor set is based on the patterns from Section 1. We include accommodation size, bedrooms, bathrooms, amenities, location, and booking-related host characteristics. These variables capture the main dimensions that are likely to influence Airbnb prices: listing size, quality, convenience, and market position. Those variables were in our view and experience the most relevant ones for a price regression.

In order for the location to be evaluable in a regression, we need to transform the latitude and longitude variables. We do this by calculating the distance to the geographic center of the dataset assuming out of experience, that there should be linear correlation between the distance to the center and the price, which gives us a single compact location measure.

```
# Replace latitude and longitude with distance to the dataset's geographic center.
# This gives one compact location measure instead of two coordinates.
lon_center = listings_2_1["longitude"].mean()
lat_center = listings_2_1["latitude"].mean()

listings_2_1["distance_center"] = np.sqrt(
    (listings_2_1["latitude"] - lat_center) ** 2 +
    (listings_2_1["longitude"] - lon_center) ** 2
```

```
)

listings_2_1 = listings_2_1.drop(columns=["latitude", "longitude"])
```

```
#Task_2_2
import statsmodels.api as sm
outcome = listings_2_1["log_price"]
regressor = listings_2_1.drop(columns=["log_price"])
regressor = sm.add_constant(regressor)

bool_cols = regressor.select_dtypes("bool").columns
regressor[bool_cols] = regressor[bool_cols].astype(int)
#print(regressor.dtypes)
olssim = sm.OLS(outcome, regressor)
olssim = olssim.fit()
olssim.summary()
```

Dep. Variable:	log_price	**R-squared:**	0.465
Model:	OLS	**Adj. R-squared:**	0.464
Method:	Least Squares	**F-statistic:**	985.4
Date:	Fri, 20 Mar 2026	**Prob (F-statistic):**	0.00
Time:	23:58:32	**Log-Likelihood:**	-5265.1
No. Observations:	9093	**AIC:**	1.055e+04
Df Residuals:	9084	**BIC:**	1.061e+04
Df Model:	8		
Covariance Type:	nonrobust		

	coef	**std err**	**t**	**P> |t|**	**[0.025**	**0.975]**
const	4.0318	0.017	236.237	0.000	3.998	4.065
accommodates	0.0770	0.003	23.520	0.000	0.071	0.083
amenities_count	0.0066	0.000	19.610	0.000	0.006	0.007
bedrooms	0.1199	0.008	14.763	0.000	0.104	0.136
host_is_superhost	0.0451	0.010	4.445	0.000	0.025	0.065
Shared_bath_status	-0.4623	0.013	-35.619	0.000	-0.488	-0.437
instant_bookable	0.1926	0.010	19.835	0.000	0.174	0.212
bathrooms_number	0.0896	0.011	8.083	0.000	0.068	0.111
distance_center	-1.5488	0.093	-16.632	0.000	-1.731	-1.366

Omnibus:	343.010	**Durbin-Watson:**	1.659
Prob(Omnibus):	0.000	**Jarque-Bera (JB):**	1004.417
Skew:	-0.088	**Prob(JB):**	7.83e-219
Kurtosis:	4.619	**Cond. No.**	699.

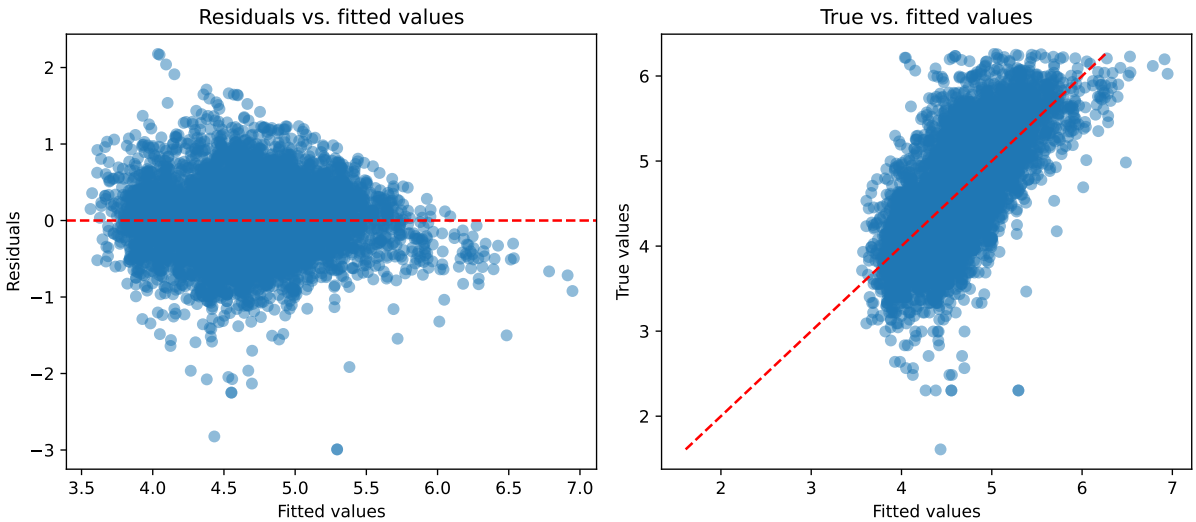
Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Note, that we carried the filtered data to exclude the top 1% of listings.

Because p<0.05, we reject the null hypothesis of normality. Since we are dealing with a log_normal distribution we have to convert the coefficient to a percentage change by using the formula: 100*(exp(coef)-1). This means that the coefficient of 0.0441 for host_is_superhost translates to a price increase of about +4.51% compared to non-superhosts, holding all other factors constant. Therefore, there is a premium for apartments with superhosts of about +4.51%. An instant bookable listing has a price increase of about +21.2% compared to non-instant bookable listings, holding all other factors constant. On the other hand the the distance to the center has a negative coefficient, which means that listings farther from the center tend to be cheaper.They tend to be -78.7% cheaper than those at the center.

An R^2 of 0,465 indicates that the model explains about 46.5% of the variance in log prices. This is a solid but not high level of explanatory power, suggesting that while the included predictors capture important factors, there is still substantial variation in prices that is not explained by the model.

```
# Task 2_3

import matplotlib.pyplot as plt

fitted = olssim.fittedvalues
residuals = olssim.resid

fig, ax = plt.subplots(1, 2, figsize=(10, 4.5))

ax[0].scatter(fitted, residuals, alpha=0.5)
ax[0].axhline(0, color='red', linestyle='--')
ax[0].set_xlabel("Fitted values")
ax[0].set_ylabel("Residuals")
ax[0].set_title("Residuals vs. fitted values")

ax[1].scatter(fitted, outcome, alpha=0.5)
ax[1].plot([outcome.min(), outcome.max()], [outcome.min(), outcome.max()], 'r--')
ax[1].set_xlabel("Fitted values")
ax[1].set_ylabel("True values")
ax[1].set_title("True vs. fitted values")

plt.tight_layout()
plt.show()
print(len(outcome))
```



9093

The residuals are centered around zero and do not show a strong systematic pattern, which supports the linear specification as a reasonable first approximation. At the same time, there is still some spread and a few visible outliers, so the model should not be treated as perfect. The fitted-vs-true plot shows that the model captures broad price differences, but it does not explain all variation in listing prices.

```
#Task 2_4 preparation
#First we used the already included RidgeCV function from sklearn, which performs Ridge regression wi
import sklearn.linear_model

ridgecv_reg = sklearn.linear_model.RidgeCV(cv=10, fit_intercept = False)
model_ridge = ridgecv_reg.fit(regressor, outcome)

print(model_ridge.coef_)
Y2_est = model_ridge.predict(regressor) # Prediction for Y2
MSE = np.mean((outcome-Y2_est)**2)
```

```
print("Mean-Squared-Error:", round(MSE, 4))
print("R^2 Score:", round(model_ridge.score(regressor, outcome), 4))
#A comparisson with the results of the following manual Ridge estimation with NumPy shows that the co
```

```
[ 4.03070231  0.07701957  0.00659014  0.11985903  0.04518235 -0.46216463
  0.19271042  0.08980038 -1.54034958]
Mean-Squared-Error: 0.1864
R^2 Score: 0.4646
```

```
# Task 2_4: manual Ridge estimation with NumPy
def ridge_estimate(X: np.ndarray, y: np.ndarray, lam: float) -> np.ndarray:
    """Compute Ridge coefficients using the closed-form formula."""
    X = np.asarray(X)
    y = np.asarray(y)
    p = X.shape[1]
    I = np.eye(p)

    beta_ridge = np.linalg.inv(X.T @ X + lam * I) @ (X.T @ y)
    return beta_ridge

def l2_norm(beta: np.ndarray) -> float:
    """Compute the Euclidean norm of the coefficient vector."""
    return np.sqrt(np.sum(beta ** 2))

lambdas = [0.01, 1, 100]
X_data = regressor.values
y_data = outcome.values

for lam in lambdas:
    beta_hat = ridge_estimate(X_data, y_data, lam)
    norm = l2_norm(beta_hat)

    print(f"Results for lambda = {lam}:")
    print(f"||beta||_2 (l2 norm): {norm:.6f}")
    print(f"Coefficients (excerpt): {beta_hat[:3]}")
    print("-" * 30)
```

```
Results for lambda = 0.01:
||beta||_2 (l2 norm): 4.351098
Coefficients (excerpt): [4.03171157 0.07699538 0.0065857 ]
------------------------------
Results for lambda = 1:
||beta||_2 (l2 norm): 4.313049
Coefficients (excerpt): [4.02085447 0.07725541 0.00663449]
------------------------------
Results for lambda = 100:
||beta||_2 (l2 norm): 3.513462
Coefficients (excerpt): [3.47407698 0.0904675  0.01103062]
------------------------------
```

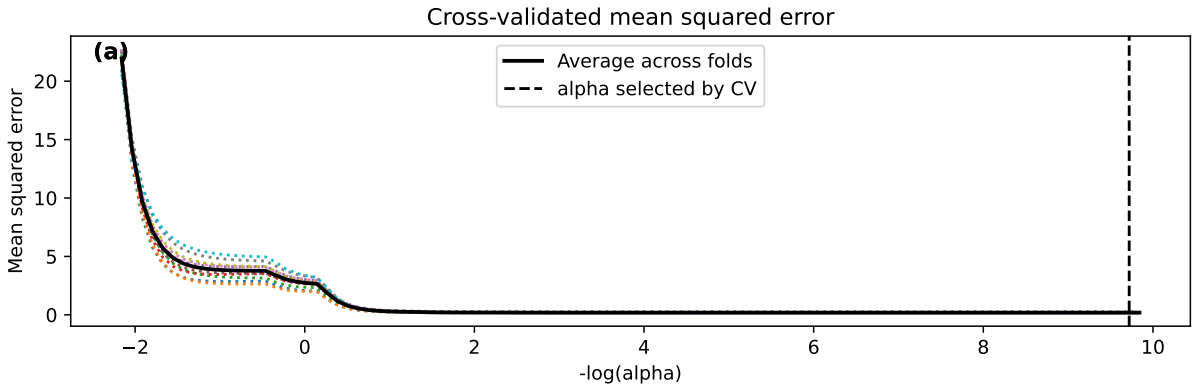
The l_2-norm || ||_2 = 3.513462 is the smallest für the Ridge regression model at lambda=100, which suggests that it has the most regularization effect among the three models. This is expected, as Ridge regression adds a penalty term to the loss function that shrinks the coefficients towards zero, thus reducing their magnitude. The Lasso regression model has a larger l_2-norm than Ridge, but still smaller than the OLS model, indicating that it also applies some regularization but allows for some coefficients to be exactly zero.

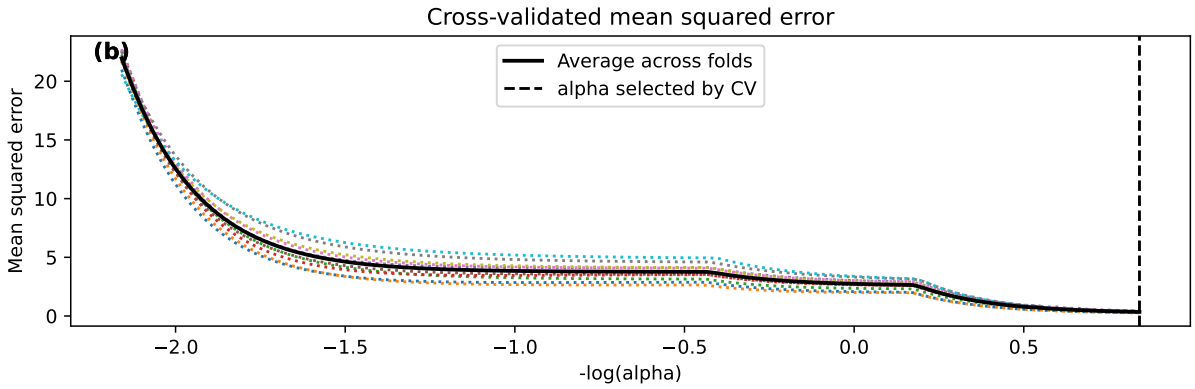
```
# Task 2_5
# fit_intercept=False is used because the regressor matrix already contains a constant.
fig, ax = plt.subplots(2, 1, figsize=(9, 6))
```

```python
def lasso_with_eps(epsilon, x_ax):
    lasso_reg = sklearn.linear_model.LassoCV(cv=10, fit_intercept=False, eps=epsilon)
    model = lasso_reg.fit(regressor, outcome)

    print("Lasso coefficients:")
    print(model.coef_)


    m_log_alphas = -np.log10(model.alphas_)
    ax[x_ax].plot(m_log_alphas, model.mse_path_, ':')
    ax[x_ax].plot(m_log_alphas, model.mse_path_.mean(axis=-1), 'k',
                  label='Average across folds', linewidth=2)
    ax[x_ax].axvline(-np.log10(model.alpha_), linestyle='--', color='k',
                     label='alpha selected by CV')
    ax[x_ax].legend()
    ax[x_ax].set_xlabel('-log(alpha)')
    ax[x_ax].set_ylabel('Mean squared error')
    ax[x_ax].set_title('Cross-validated mean squared error')
    ax[x_ax].axis('tight')

    ax[0].text(0.02, 0.98, "(a)", transform=ax[0].transAxes,
            ha="left", va="top", fontsize=12, fontweight="bold")
    ax[1].text(0.02, 0.98, "(b)", transform=ax[1].transAxes,
            ha="left", va="top", fontsize=12, fontweight="bold")

    zero_mask = model.coef_ == 0
    zero_features = regressor.columns[zero_mask]
    non_zero_features = regressor.columns[~zero_mask]

    print("Alpha selected via cross-validation:", round(model.alpha_, 13))
    print("List of Zero-coefficient features:")
    print(list(zero_features))
    print("List of Non-zero-coefficient features:")
    print(list(non_zero_features))
    print("L2 norm of coefficients:", np.linalg.norm(model.coef_, 2))
    print("-" * 30)

lasso_with_eps(1e-12, 0)
lasso_with_eps(1e-3, 1)
plt.tight_layout()
plt.show()
```

```
Lasso coefficients:
[ 4.03182399  0.07699268  0.0065852   0.11988275  0.04511646 -0.46232801
  0.19259124  0.08959394 -1.54878601]
Alpha selected via cross-validation: 1.906e-10
List of Zero-coefficient features:
[]
List of Non-zero-coefficient features:
['const', 'accommodates', 'amenities_count', 'bedrooms', 'host_is_superhost', 'Shared_bath_status',
L2 norm of coefficients: 4.3515040011543045
------------------------------
Lasso coefficients:
[ 3.05254322  0.18991629  0.02634185  0.          0.         -0.
  0.          0.03199987  0.        ]
Alpha selected via cross-validation: 0.1442021152149
List of Zero-coefficient features:
['bedrooms', 'host_is_superhost', 'Shared_bath_status', 'instant_bookable', 'distance_center']
```

```
List of Non-zero-coefficient features:
['const', 'accommodates', 'amenities_count', 'bathrooms_number']
L2 norm of coefficients: 3.058726239682447
------------------------------
```





The choice of epsilon in LassoCV directly affects the selected regularization strength and therefore the resulting model. With the default setting (epsilon = 1e−3), the selected is relatively large, which leads to strong regularization. In this case, several coefficients are pushed exactly to zero, specifically bedrooms, host_is_superhost, Shared_bath_status, instant_bookable, and distance_center, resulting in a sparse model that retains only a small subset of predictors.

When is reduced to a very small value (around 1e−12), the selected becomes extremely small. As a result, Lasso behaves almost identically to OLS: coefficients are barely shrunk, no variables are eliminated, and the l_2-norm remains close to the OLS solution. This indicates that there is effectively no regularization.

In comparison, OLS includes all variables without any shrinkage, while Ridge reduces coefficient magnitudes as lambda increases but never sets them to zero. Lasso differs in that it can both shrink coefficients and eliminate variables entirely. Overall, the results show that Lasso transitions from an OLS-like model without sparsity to a sparse model with variable selection as the regularization strength increases, with epsilon indirectly controlling this behavior through its influence on .

For the Minimum of the Average MSE to show, eps must be set to a the value of ~1e-12. This results in a very small alpha, which is the regalurization strength. The small alpha indicates that the model is very close to an OLS regression, which is consistent with the fact that the lasso coefficients are not heavily shrunk towards zero. The fact that some coefficients are exactly zero suggests that the lasso has identified certain features as less relevant for predicting the outcome, while still allowing other features to have non-zero coefficients. The extension to eps=1e-12 would result therefore in zero "Zero-coefficients" and only Non-zero coefficients.

```
#Aufgabe 2_6
from sklearn.model_selection import KFold
from sklearn.metrics import mean_squared_error
X = regressor.values
y = outcome.values
kf = KFold(n_splits=5, shuffle=True, random_state=12883823)
```

```
rmse_ols = []
rmse_ridge = []
rmse_lasso = []
for train, test in kf.split(X):
    X_train, X_test, y_train, y_test = X[train], X[test], y[train], y[test]

    # OLS
    ols_model_2_6 = sm.OLS(y_train, X_train).fit()
    y_pred_ols_2_6 = ols_model_2_6.predict(X_test)
    rmse_ols.append(np.sqrt(mean_squared_error(y_test, y_pred_ols_2_6)))

    # Ridge
    ridgecv_reg_2_6 = sklearn.linear_model.RidgeCV(cv=5, fit_intercept = False).fit(X_train, y_train
    Y2_est_2_6 = ridgecv_reg_2_6.predict(X_test)
    rmse_ridge.append(np.sqrt(mean_squared_error(y_test, Y2_est_2_6)))

    # Lasso
    lasso_reg_2_6 = sklearn.linear_model.LassoCV(cv=5,fit_intercept =False).fit(X_train, y_train)
    y_pred_lasso_2_6 = lasso_reg_2_6.predict(X_test)
    rmse_lasso.append(np.sqrt(mean_squared_error(y_test, y_pred_lasso_2_6)))

print("Average RMSE OLS:", np.mean(rmse_ols))
print("Average RMSE Ridge:", np.mean(rmse_ridge))
print("Average RMSE Lasso:", np.mean(rmse_lasso))
```

```
Average RMSE OLS: 0.43255376867743134
Average RMSE Ridge: 0.43255435670231507
Average RMSE Lasso: 0.5890117589685507
```

OLS and Ridge achieve nearly identical out-of-sample performance, indicating that regularization does not improve predictive accuracy. Lasso performs substantially worse due to aggressive variable selection, which removes relevant predictors and leads to underfitting. The inferior performance of Lasso indicates that the underlying model is not sparse. Many predictors contribute to the outcome, so enforcing sparsity removes relevant information and increases prediction error. In contrast, OLS and Ridge retain all variables, which better reflects the data structure.

Task 2.7: The regression results suggest that listing size and quality are central for pricing. Listings that accommodate more guests, offer more bedrooms and bathrooms, and provide more amenities tend to be more expensive. Location also matters: being closer to the city centre is generally associated with higher prices. In addition, host-related features such as superhost status and instant booking appear to be linked to a price premium, although these effects are smaller than the effects of size and location. For a new host, the main advice would be to benchmark the listing against similar nearby properties and then adjust for concrete characteristics such as capacity, equipment, and booking convenience. A host should not simply copy the average market price. Instead, pricing should reflect the local area and the quality of the offering. Small improvements in presentation and amenities may justify a moderate premium, but large pricing mistakes are more likely to come from ignoring location and listing size.

```
# Task 2.8
model_2_8 = sm.QuantReg(outcome, regressor)
res_2_8 = model_2_8.fit(q=0.5)
res_2_8.summary().tables[1]
```

	coef	std err	t	P> |t|	[0.025	0.975]
const	4.0375	0.021	196.459	0.000	3.997	4.078
accommodates	0.0822	0.004	20.859	0.000	0.074	0.090
amenities_count	0.0066	0.000	16.349	0.000	0.006	0.007
bedrooms	0.1095	0.010	11.201	0.000	0.090	0.129
host_is_superhost	0.0466	0.012	3.811	0.000	0.023	0.071
Shared_bath_status	-0.4556	0.016	-29.152	0.000	-0.486	-0.425
instant_bookable	0.1924	0.012	16.457	0.000	0.169	0.215
bathrooms_number	0.0964	0.013	7.221	0.000	0.070	0.123
distance_center	-1.7936	0.112	-15.996	0.000	-2.013	-1.574

The largest differences between the OLS and quantile regression estimates are observed for distance to the city centre, which shows a substantially stronger negative effect in the median regression. Smaller but noticeable differences also appear for accommodates and bathrooms, which have slightly larger coefficients in the quantile model. In contrast, most other coefficients remain very similar across the two specifications. Importantly, the statistical significance of all variables remains unchanged: all coefficients are highly significant in both models, and their signs are consistent. This suggests that the main relationships are robust, although the strength of some effects varies depending on whether the mean or the median of the price distribution is considered.

```
# Task 2.9
# H0: beta_host_is_superhost = 0
# H1: beta_host_is_superhost != 0
ols_2_9 = sm.OLS(outcome, regressor).fit(cov_type="HC3")
ols_2_9.summary().tables[1]

print(pd.concat([ols_2_9.params, ols_2_9.tvalues, ols_2_9.pvalues], axis=1,
                keys=["Coefficient", "t-Statistic", "p-Value"]).loc["host_is_superhost"])

print(ols_2_9.conf_int(0.05).rename(columns={0: "2.5%", 1: "97.5%"}).loc["host_is_superhost"])
print(f"Percentage premium for superhost listings: {(np.exp(ols_2_9.params['host_is_superhost']) - 1
```

```
Coefficient    0.045116
t-Statistic    4.580941
p-Value        0.000005
Name: host_is_superhost, dtype: float64
2.5%     0.025813
97.5%    0.064420
Name: host_is_superhost, dtype: float64
Percentage premium for superhost listings: 4.61%
```

We investigate whether being a superhost has a statistically significant effect on Airbnb listing prices. The null hypothesis is that there is no effect of superhost status on log price (H : _superhost = 0), while the alternative hypothesis states that there is an effect (H : _superhost 0).

Using HC3-robust standard errors, the estimated coefficient is 0.045, with a t-statistic of 4.58 and a p-value of 0.000005. This provides strong evidence against the null hypothesis. The 95% confidence interval based on HC3 standard errors ranges from 0.0258 to 0.0644, which does not include zero and therefore supports the statistical significance of the effect.

In real-world terms, the coefficient implies that superhost listings are associated with approximately 4.6% higher prices, holding other factors constant. This suggests that superhost status is valued in the Berlin Airbnb market, possibly reflecting higher trust, better service quality, or more positive guest experiences.

Task 2.10 Heteroscedasticity-robust standard errors are important because the variance of the error term is unlikely to be constant across Airbnb listings. In particular, more expensive listings tend to exhibit greater price variation than cheaper ones, leading to heteroscedasticity. While OLS coefficient estimates remain unbiased in this case, the standard errors become inconsistent. If heteroscedasticity were ignored, statistical inference would be unreliable, as standard errors are often underestimated, resulting in overly small p-values and potentially false significance.

The estimated coefficient on the superhost variable should be interpreted as an association rather than a causal effect. Superhosts may differ systematically from other hosts in ways not fully captured by the model. For example, unobserved factors such as listing quality, interior design, location-specific attractiveness, or guest ratings may influence both the likelihood of being a superhost and the listing price, leading to omitted variable bias.

Section 3

```python
# Task 3.1
import pandas as pd

# Load the training data.
X_train = pd.read_csv("../Daten//X_train.csv", index_col=0)
y_train = pd.read_csv("../Daten//y_train.csv", index_col=0)

n_obs, n_variables = X_train.shape
print(f"Observations: {n_obs}, Variables: {n_variables}")

default_fraction = y_train.mean().values[0]
print(f"Fraction of defaults: {default_fraction:.2%}")

summary_stats = X_train.describe().loc[['mean', 'std', 'min', 'max']]
print(summary_stats)
```

```
Observations: 5000, Variables: 20
Fraction of defaults: 35.02%
      annual_income        age  employment_years  debt_to_income  \
mean   50309.452058  43.626200          5.733020        0.166050
std    29349.199007  12.364216          4.166518        0.159139
min     5473.630000  22.000000          0.100000        0.007300
max   338861.400000  65.000000         40.000000        1.500000

      credit_score  credit_utilization  num_open_accounts  \
mean    678.794200            0.280992           4.948400
std      75.449405            0.156996           2.193422
min     434.000000            0.006200           0.000000
max     850.000000            0.869900          13.000000

      num_credit_inquiries   loan_amount  interest_rate  loan_term_months  \
mean              2.473600  11827.435264       8.402370         37.152000
std               1.568373   6533.911480       2.097997         14.089961
min               0.000000   1812.350000       2.000000         12.000000
max              10.000000  77433.770000      16.390000         60.000000

      home_ownership  months_since_delinquency  total_credit_lines  \
mean        0.415200                  8.968600            7.933600
std         0.492806                 16.673632            2.810973
min         0.000000                  0.000000            0.000000
max         1.000000                156.000000           21.000000

      revolving_balance  savings_balance  payment_history_score  \
mean       5.747878e+03      1804.155792              75.535580
std        2.421299e+04      2281.415292              13.619852
min        1.063000e+01        19.780000              16.800000
max        1.436268e+06     30942.310000             100.000000

      num_dependents  previous_defaults  monthly_installment
mean        0.988200           0.092400           436.871034
std         0.994715           0.289619           351.666441
min         0.000000           0.000000            53.670000
max         7.000000           1.000000          4595.210000
```

Distance-based methods such as KNN and SVM are sensitive to the scale of the input variables. Features like annual income, loan amount, and revolving balance have much larger numerical ranges than binary indicators or ratios. Without scaling, those large-scale variables would dominate the distance calculations.

For that reason, standardization is necessary before applying methods that rely on distances or margins in feature space.

```
# Task 3.2
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

# Standardize the features.
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)

# Fit logistic regression.
log_reg = LogisticRegression()
log_reg.fit(X_train_scaled, y_train.values.ravel())

# Report training accuracy.
y_pred = log_reg.predict(X_train_scaled)
train_accuracy = accuracy_score(y_train, y_pred)
print(f"Training Accuracy: {train_accuracy:.4f}")

# Rank features by absolute coefficient size.
coef_df = pd.DataFrame({
    'Feature': X_train.columns,
    'Coefficient': log_reg.coef_[0],
    'Abs_Coefficient': abs(log_reg.coef_[0])
}).sort_values(by='Abs_Coefficient', ascending=False)

print("Top 3 features by absolute coefficient:")
print(coef_df.head(3))
```

```
Training Accuracy: 0.7298
Top 3 features by absolute coefficient:
               Feature  Coefficient  Abs_Coefficient
3       debt_to_income     0.853965         0.853965
5   credit_utilization     0.378353         0.378353
18   previous_defaults     0.368845         0.368845
```

After standardization, the coefficients can be compared more directly because all variables are measured on the same scale. A positive coefficient indicates that higher values of the feature are associated with a higher probability of default, while a negative coefficient points in the opposite direction. The variables with the largest absolute coefficients are therefore the strongest predictors in the logistic model. The features with the largest absolute coefficients are debt_to_income, credit_utilization, and previous_defaults. They indicate that default risk is mainly driven by a borrower's financial burden, current credit usage, and past behavior. A high debt-to-income ratio is the strongest predictor, suggesting that individuals with higher relative debt are much more likely to default. High credit utilization further increases risk by indicating financial strain, while previous defaults highlight that past repayment behavior is a strong indicator of future default.

```
# Task 3.3
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

# Split the data into training and validation sets.
X_train_k, X_val_k, y_train_k, y_val_k = train_test_split(
    X_train_scaled, y_train.values.ravel(), test_size=0.2, random_state=12883823
)

# Test different values of k and different weighting schemes.
k_values = [5, 20, 50]
```

```
weight_options = ['uniform', 'distance']

results = []

for k in k_values:
    for weight in weight_options:
        knn = KNeighborsClassifier(n_neighbors=k, weights=weight)
        knn.fit(X_train_k, y_train_k)

        accuracy = knn.score(X_val_k, y_val_k)
        error_rate = 1 - accuracy

        results.append({
            'k': k,
            'weights': weight,
            'error_rate': error_rate
        })

knn_results_df = pd.DataFrame(results)
print(knn_results_df)
```

```
    k   weights  error_rate
0   5   uniform       0.311
1   5  distance       0.311
2  20   uniform       0.312
3  20  distance       0.310
4  50   uniform       0.309
5  50  distance       0.308
```

The results show only small differences across the different KNN specifications. The best performance is achieved with k = 50 and distance-based weighting, resulting in an error rate of 0.308. In general, larger values of k perform slightly better than smaller ones. This is consistent with the idea that larger k values produce smoother and more stable decision boundaries, reducing sensitivity to noise in the data. Distance weighting also leads to marginal improvements, as closer neighbors receive more influence than distant ones. KNN may struggle with this dataset due to the relatively high dimensionality (20 features). In high-dimensional spaces, distances between observations become less informative, which makes it harder to identify truly similar neighbors. As a result, the classification performance remains limited even after tuning k and the weighting scheme.

```
# Task 3.4
from sklearn.svm import SVC

# Fit SVM with a linear kernel.
svm_linear = SVC(kernel='linear')
svm_linear.fit(X_train_k, y_train_k)
acc_linear = svm_linear.score(X_val_k, y_val_k)

# Fit SVM with an RBF kernel for different values of C.
c_values = [0.1, 1, 10]
rbf_results = {}

for c in c_values:
    svm_rbf = SVC(kernel='rbf', C=c)
    svm_rbf.fit(X_train_k, y_train_k)
    acc_rbf = svm_rbf.score(X_val_k, y_val_k)
    rbf_results[c] = acc_rbf

# Logistic regression benchmark.
log_reg_val = LogisticRegression()
```

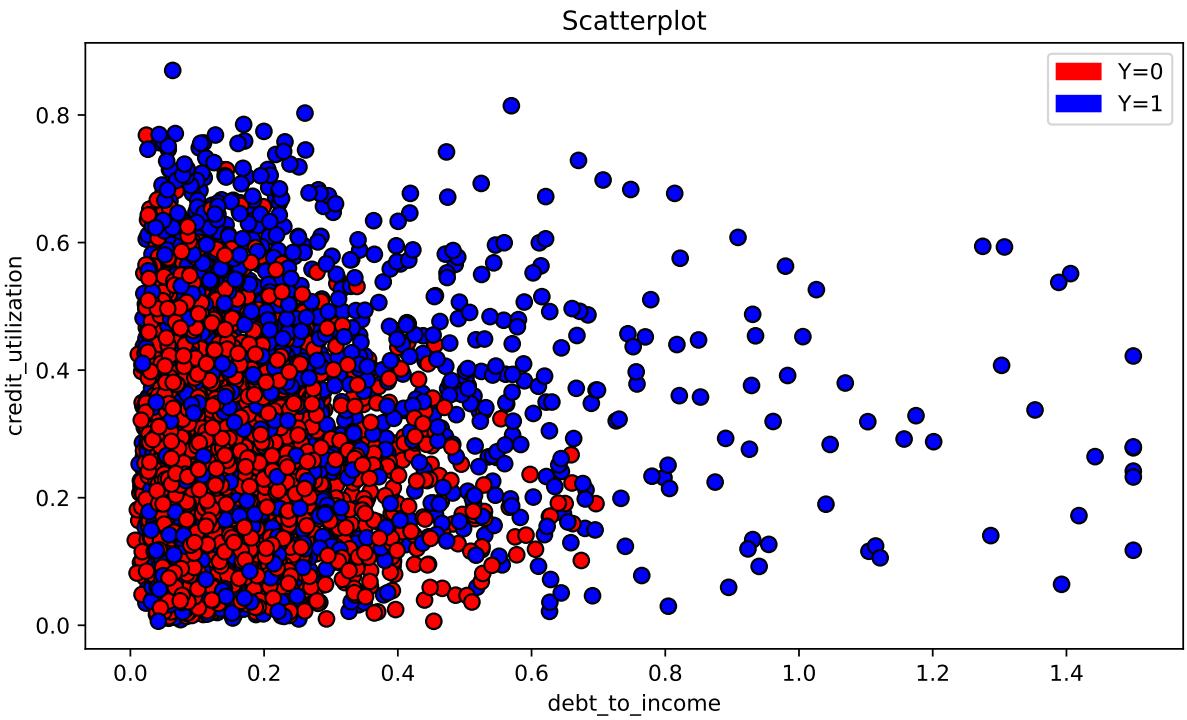
```
log_reg_val.fit(X_train_k, y_train_k)
acc_log_reg = log_reg_val.score(X_val_k, y_val_k)

print(f"Logistic Regression Accuracy: {acc_log_reg:.4f}")
print(f"SVM Linear Kernel Accuracy: {acc_linear:.4f}")
for c, acc in rbf_results.items():
    print(f"SVM RBF Kernel (C={c}) Accuracy: {acc:.4f}")
```

```
Logistic Regression Accuracy: 0.7060
SVM Linear Kernel Accuracy: 0.6910
SVM RBF Kernel (C=0.1) Accuracy: 0.6970
SVM RBF Kernel (C=1) Accuracy: 0.7360
SVM RBF Kernel (C=10) Accuracy: 0.7060
```

Tthe RBF kernel performs better than the linear kernel. If this is the case, it suggests that the relationship between the predictors and default risk is not purely linear. In that case, a curved decision boundary can separate the classes more effectively than a straight line. This would be plausible if default risk depends on interactions, nonlinear effects, or threshold behavior in the borrower characteristics.

```
# Task 3.5
import numpy as np
X = X_train[["debt_to_income", "credit_utilization"]].values
y = y_train.values.ravel()

logisticRegr = LogisticRegression(l1_ratio=0, solver='liblinear')
model_3_5_1 = logisticRegr.fit(X, y)
y_score = model_3_5_1.predict(X)
print('Error rate:', np.mean(np.not_equal(model_3_5_1.predict(X), y)))

import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
import matplotlib.patches as mpatches
plot_symbol_size = 50
cmap_bold = ListedColormap(['#FF0000', '#0000FF'])
plt.figure(figsize=(9, 5))
plt.scatter(X[:, 0], X[:, 1], s=plot_symbol_size, c=y, cmap=cmap_bold, edgecolor='black')
plt.xlabel("debt_to_income")
plt.ylabel("credit_utilization")
plt.title("Scatterplot")
patch0 = mpatches.Patch(color='#FF0000', label='Y=0')
patch1 = mpatches.Patch(color='#0000FF', label='Y=1')
plt.legend(handles=[patch0, patch1])
plt.show()
```

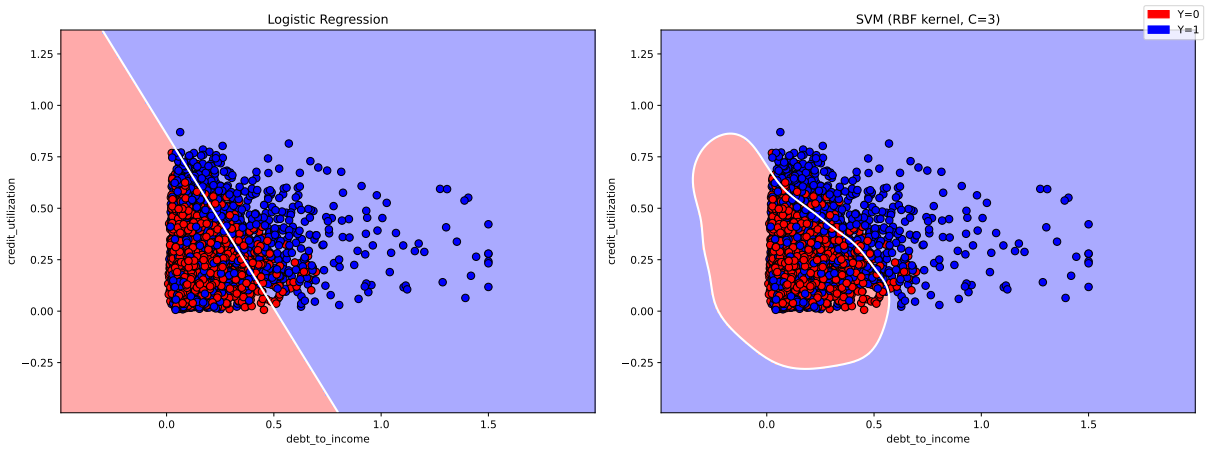
```
Error rate: 0.2874
```




```
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
import matplotlib.patches as mpatches
from sklearn import svm

Y = y


cmap_light = ListedColormap(['#FFAAAA', '#AAAAFF'])
cmap_bold  = ListedColormap(['#FF0000', '#0000FF'])

mesh_step_size = 0.01
plot_symbol_size = 50

x_min, x_max = X[:, 0].min() - .5, X[:, 0].max() + .5
y_min, y_max = X[:, 1].min() - .5, X[:, 1].max() + .5
xx, yy = np.meshgrid(np.arange(x_min, x_max, mesh_step_size),
                     np.arange(y_min, y_max, mesh_step_size))

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Logistic Regression
Z_lr = model_3_5_1.predict(np.c_[xx.ravel(), yy.ravel()])
D_lr = model_3_5_1.decision_function(np.c_[xx.ravel(), yy.ravel()])
Z_lr = Z_lr.reshape(xx.shape)
D_lr = D_lr.reshape(xx.shape)

axes[0].pcolormesh(xx, yy, Z_lr, cmap=cmap_light)
axes[0].scatter(X[:, 0], X[:, 1], s=plot_symbol_size, c=Y,
                cmap=cmap_bold, edgecolor='black')
axes[0].contour(xx, yy, D_lr, levels=[0], colors='w', linewidths=2)

axes[0].set_title("Logistic Regression")
axes[0].set_xlabel('debt_to_income')
```

```
axes[0].set_ylabel('credit_utilization')
axes[0].set_xlim(xx.min(), xx.max())
axes[0].set_ylim(yy.min(), yy.max())

# SVM
supvecma = svm.SVC(kernel='rbf', C=3)
model_3_5_2 = supvecma.fit(X, Y)

print('Error rate:', np.mean(np.not_equal(model_3_5_2.predict(X), y)))

Z_svm = model_3_5_2.predict(np.c_[xx.ravel(), yy.ravel()])
D_svm = model_3_5_2.decision_function(np.c_[xx.ravel(), yy.ravel()])
Z_svm = Z_svm.reshape(xx.shape)
D_svm = D_svm.reshape(xx.shape)

axes[1].pcolormesh(xx, yy, Z_svm, cmap=cmap_light)
axes[1].scatter(X[:, 0], X[:, 1], s=plot_symbol_size, c=Y,
                cmap=cmap_bold, edgecolor='black')
axes[1].contour(xx, yy, D_svm, levels=[0], colors='w', linewidths=2)

axes[1].set_title("SVM (RBF kernel, C=3)")
axes[1].set_xlabel('debt_to_income')
axes[1].set_ylabel('credit_utilization')
axes[1].set_xlim(xx.min(), xx.max())
axes[1].set_ylim(yy.min(), yy.max())

patch0 = mpatches.Patch(color='#FF0000', label='Y=0')
patch1 = mpatches.Patch(color='#0000FF', label='Y=1')
fig.legend(handles=[patch0, patch1], loc='upper right')

plt.tight_layout()
plt.show()
```

```
Error rate: 0.2748
```



This comparison shows that logistic regression is relatively rigid and may underfit when the relationship between debt-to-income and credit utilization is nonlinear, since it produces a linear decision boundary. The RBF-SVM, on the other hand generates a nonlinear and more flexible boundary, that adapts to the structure of the data and better captures regions with higher default risk.

```
#Aufgabe 3_6
from sklearn.ensemble import RandomForestClassifier
#visualize top 5 values
randforest = RandomForestClassifier(max_depth=5, random_state=0)
```

```python
model_3_6 = randforest.fit(X_train, y_train.values.ravel())
print("Error rate:", np.mean(model_3_6.predict(X_train) != y_train.values.ravel()))
importance=pd.DataFrame([model_3_6.feature_importances_],index=["importance"], columns=X_train.colum
print(importance.head())

#Function to play around
def rand_for_var(n_estimat, max_dep):
    randforest = RandomForestClassifier(max_depth=max_dep, random_state=0, n_estimators=n_estimat)
    model_3_6 = randforest.fit(X_train, y_train.values.ravel())
    print("Error rate:", np.mean(model_3_6.predict(X_train) != y_train.values.ravel()))
    return

for n in [1, 10, 100]:
    print("-"*30)
    print(f"n_estimators = {n}")
    print(rand_for_var(n, 5))

for max in [5, 20, 50]:
    print("-"*30)
    print(f"max_depth = {max}")
    print(rand_for_var(10, max))
```

```
Error rate: 0.2366
                       importance
credit_utilization       0.240613
debt_to_income           0.237807
previous_defaults        0.109675
annual_income            0.088058
payment_history_score    0.067746
------------------------------
n_estimators = 1
Error rate: 0.2684
None
------------------------------
n_estimators = 10
Error rate: 0.2432
None
------------------------------
n_estimators = 100
Error rate: 0.2366
None
------------------------------
max_depth = 5
Error rate: 0.2432
None
------------------------------
max_depth = 20
Error rate: 0.0348
None
------------------------------
max_depth = 50
Error rate: 0.018
None
```

The leading variables from the random forest importance ranking are broadly in line with the earlier logistic-regression results, even if the exact order differs. This is reassuring because two different model classes point to a similar set of important predictors.

```python
# Task 3.7a
from sklearn.metrics import log_loss

# Use scaled predictors for the model comparison.
X = X_train_scaled
y = y_train.values.ravel()

kf = KFold(n_splits=5, shuffle=True, random_state=12883823)
acc_log_reg_3_7 = []
acc_knn_3_7 = []
acc_svm_3_7 = []
acc_rf_3_7 = []

ll_log_reg_3_7 = []
ll_knn_3_7 = []
ll_svm_3_7 = []
ll_rf_3_7 = []

for train, test in kf.split(X):
    X_train_3_7, X_test_3_7, y_train_3_7, y_test_3_7 = X[train], X[test], y[train], y[test]

    model_3_7_1 = LogisticRegression().fit(X_train_3_7, y_train_3_7)
    acc_log_reg_3_7.append(accuracy_score(y_test_3_7, model_3_7_1.predict(X_test_3_7)))
    ll_log_reg_3_7.append(log_loss(y_test_3_7, model_3_7_1.predict_proba(X_test_3_7)))

    model_3_7_2 = KNeighborsClassifier(n_neighbors=50, weights='distance').fit(X_train_3_7, y_train_
    acc_knn_3_7.append(accuracy_score(y_test_3_7, model_3_7_2.predict(X_test_3_7)))
    ll_knn_3_7.append(log_loss(y_test_3_7, model_3_7_2.predict_proba(X_test_3_7)))

    model_3_7_3 = SVC(kernel='rbf', C=1, probability=True).fit(X_train_3_7, y_train_3_7)
    acc_svm_3_7.append(accuracy_score(y_test_3_7, model_3_7_3.predict(X_test_3_7)))
    ll_svm_3_7.append(log_loss(y_test_3_7, model_3_7_3.predict_proba(X_test_3_7)))

    # For the random forest classifier, we limited the depth of 11 in order to in order to combat ov
    model_3_7_4 = RandomForestClassifier(max_depth=11, random_state=0).fit(X_train_3_7, y_train_3_7)
    acc_rf_3_7.append(accuracy_score(y_test_3_7, model_3_7_4.predict(X_test_3_7)))
    ll_rf_3_7.append(log_loss(y_test_3_7, model_3_7_4.predict_proba(X_test_3_7)))

print(f"Average Accuracy Logistic Regression: {np.mean(acc_log_reg_3_7):.2%}")
print(f"Average Accuracy KNN: {np.mean(acc_knn_3_7):.2%}")
print(f"Average Accuracy SVM: {np.mean(acc_svm_3_7):.2%}")
print(f"Average Accuracy Random Forest: {np.mean(acc_rf_3_7):.2%}")
print(f"Average Log Loss Logistic Regression: {np.mean(ll_log_reg_3_7):.2f}")
print(f"Average Log Loss KNN: {np.mean(ll_knn_3_7):.2f}")
print(f"Average Log Loss SVM: {np.mean(ll_svm_3_7):.2f}")
print(f"Average Log Loss Random Forest: {np.mean(ll_rf_3_7):.2f}")
```

```
Average Accuracy Logistic Regression: 72.86%
Average Accuracy KNN: 71.36%
Average Accuracy SVM: 74.38%
Average Accuracy Random Forest: 74.82%
Average Log Loss Logistic Regression: 0.57
Average Log Loss KNN: 0.58
Average Log Loss SVM: 0.54
Average Log Loss Random Forest: 0.54
```

The cross-validated comparison points to Random Forest as the strongest model in this notebook. It achieves the best overall performance not only in terms of accuracy but also in terms of log loss. That matters because accuracy alone ignores how confident the predicted probabilities are. Two models can have

the same number of correct classifications, while one produces much more sensible probability estimates. Looking at both measures therefore gives a fairer comparison. In our results, Random Forest performs best on both dimensions, so it is the most convincing choice for the final prediction task.

```
# Task 3.8
X_test_3_8 = pd.read_csv("../Daten//X_test.csv", index_col=0)
print(X_test_3_8)
# Random Forest performed best in Task 3.7, so we use it for the final predictions.
y_test_pred = model_3_6.predict_proba(X_test_3_8)

model_3_6.classes_
predictions = y_test_pred[:, 1]
print(predictions)
pd.Series(predictions).to_csv("../Daten//predictions.csv", header=["probability"], index_label="id")
```

```
      annual_income   age  employment_years  debt_to_income  credit_score  \
5000       61147.74  31.0               5.6          0.1838         643.0
5001       52349.60  23.0               3.2          0.1138         708.0
5002       33045.87  53.0               5.2          0.1785         634.0
5003      108675.59  45.0               2.8          0.0517         642.0
5004       32740.92  40.0               1.8          0.1445         727.0
...             ...   ...               ...             ...           ...
7495       25029.34  36.0               2.0          0.1550         657.0
7496       50085.90  60.0               5.2          0.0828         745.0
7497       56853.36  57.0              10.8          0.0940         760.0
7498       41850.21  65.0               6.1          0.0846         702.0
7499       75377.50  31.0               4.3          0.1191         682.0

      credit_utilization  num_open_accounts  num_credit_inquiries  \
5000              0.2592                  5                     4
5001              0.2340                  5                     2
5002              0.5987                  7                     3
5003              0.4714                  3                     1
5004              0.1429                  9                     3
...                  ...                ...                   ...
7495              0.1679                  6                     1
7496              0.1610                  4                     3
7497              0.2588                  8                     1
7498              0.2760                  6                     3
7499              0.4547                  5                     4

      loan_amount  interest_rate  loan_term_months  home_ownership  \
5000     11384.48           6.58                60               1
5001     15651.77           7.89                36               0
5002      4808.37           7.57                48               0
5003     10038.49           9.85                24               1
5004     12438.32           6.75                36               0
...           ...            ...               ...             ...
7495      6927.42           8.74                60               1
7496     15795.46           6.56                60               1
7497     13668.83           7.03                36               0
7498     12976.85           9.71                60               1
7499     13719.46           3.96                24               0

      months_since_delinquency  total_credit_lines  revolving_balance  \
5000                       0.0                   7           35675.38
5001                       1.0                   5             344.44
5002                       0.0                  11           18755.41
5003                       6.0                   8             275.22
```

```
5004                      2.0                   9             585.23
...                       ...                 ...                ...
7495                      4.0                   9            9015.97
7496                      0.0                   6            1797.19
7497                     10.0                  12            1158.30
7498                      0.0                  10            1063.22
7499                      0.0                   8            7615.61

      savings_balance  payment_history_score  num_dependents  \
5000           547.75                   87.5               1
5001          6559.58                   63.7               2
5002           304.39                   78.6               2
5003          2741.28                   66.0               0
5004          2850.75                   55.2               1
...               ...                    ...             ...
7495          1173.62                   61.2               3
7496           566.63                   95.2               0
7497           764.20                   52.7               2
7498          1236.85                   42.7               0
7499          1559.44                   53.4               0

      previous_defaults  monthly_installment
5000                  0               223.18
5001                  0               489.68
5002                  0               116.42
5003                  0               462.53
5004                  1               382.64
...                 ...                  ...
7495                  0               142.93
7496                  0               309.50
7497                  0               422.24
7498                  1               273.87
7499                  1               595.52

[2500 rows x 20 columns]
[0.31185738 0.24534995 0.61337942 ... 0.29596897 0.48015116 0.54207203]
```